Labo3: Partitioning the grid into warps and reducing thread divergence

Arthur Valentin, Joeri Sevens

November 20, 2024

Overview

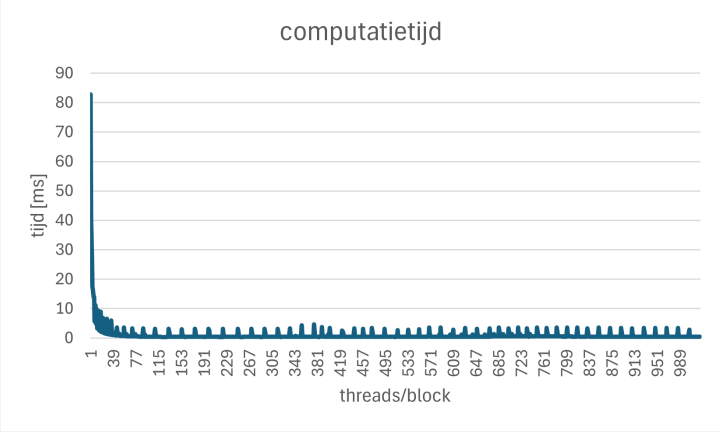
Dit labo richt zich op het gebruik van optimalisatietechnieken binnen GPU-programmering, met specifieke aandacht voor warp-partitionering en het minimaliseren van thread divergence.

In deel 1 wordt een kernel geïmplementeerd die een afbeelding inverteert door de kleurcomponenten (R, G, B) te transformeren, waarbij de invloed van het aantal threads (volledige versus gedeeltelijke warps) op de rekentijd wordt geanalyseerd.

In deel 2 wordt de kernel uitgebreid om een niet-lineaire transformatie toe te passen op de R-kleurcomponent, met als doel te begrijpen hoe thread divergence de prestaties beïnvloedt.

Methodology

We begonnen met het implementeren van een code waarin het aantal threads binnen een blok geleidelijk werd verhoogd. Zodra een blok zijn maximale capaciteit aan threads bereikte, werden de extra threads toegewezen aan de volgende blokken, waarbij dit proces werd voortgezet totdat we aan het maximale aantal threads per block zaten (1023). De grafiek hiervan is te zien in grafiek 1

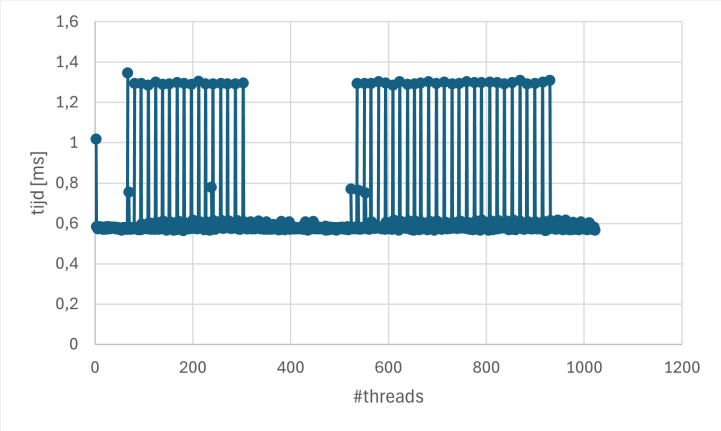


grafiek 1: tijd in ms als functie van het aantal threads per block

Voor de tweede methode lag de focus op het onderzoeken van thread divergence en het gebruik van striding. Hierbij duurde de berekening van de rode kleurcomponent langer dan de berekeningen van de blauwe en groene kleurcomponenten, doordat hier een modulo-operatie in zit. Dit zorgt voor thread divergence binnen de warp, aangezien de threads die de groene en blauwe component berekenen eerder klaar zijn en op de rode component moeten wachten. Er is bovendien nog een extra divergence aanwezig als niet elk blok met een veelvoud van 32 threads is gevuld, aangezien elke warp uit 32 threads bestaat

Eerst hebben we opnieuw de threads per blok laten stijgen en niets te doen tegen de divergence binnen een warp, zie code 2. Hierbij merkten we op dat de tijd wel iets hoger lag dan bij de vorige oefening.

Vervolgens hebben we geprobeerd om alle drie de componenten door één thread te laten uitvoeren, zodat in theorie elke thread ongeveer even lang bezig is. Bij deze code hebben we ook striding toegevoegd door het totaal aantal beschikbare threads van 1 tot 100.000 te laten stijgen. Hierbij werd elke blok steeds volledig opgevuld voordat we naar het volgende blok gingen. De resultaten hiervan zijn in de volgende grafiek te zien.



grafiek 2: tijd in ms als functie van het aantal threads

Als laatste hebben we een code geschreven die eerst alle rode componenten verwerkt, daarna alle groene en als laatste de blauwe componenten. Dit zorgt ervoor dat zoveel mogelijk berekeningen die even lang duren, samen zitten. We hadden dit ook kunnen doen door onze array te herordenen, zodat we op het eerste deel van de array (de rode componenten) de rode berekeningen doen en op het tweede en derde deel van de array (groen en blauw) de simpelere berekeningen, maar we dachten dat dit veel overhead voor de CPU zou veroorzaken. Echter, hebben we dit later in labo 4 toegepast voor coalesced en uncoalesced toegang.

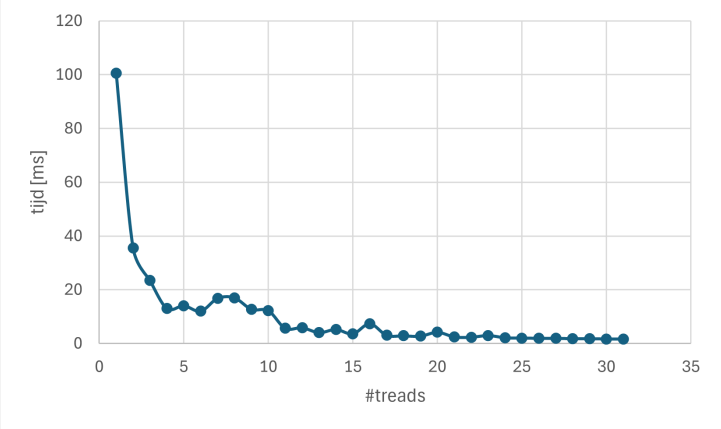
Conclusion

Assess how the number of threads (full/partial warps) influences the computation time

Uit grafiek 1 kunnen we concluderen dat de rekentijd afneemt naarmate het aantal threads per blok toeneemt. Bovendien is er een merkbare verhoging na elke 32ste blok (zie Grafiek 3), omdat er dan een warp ontstaat die slechts 1 thread benut en dus 31 threads onbenut laat. Deze resultaten onderstrepen het belang van een efficiënte toewijzing van threads voor optimale prestaties.

Evaluate how the execution time changes with / without thread divergence.

Theoretisch zou minder thread-divergentie moeten leiden tot een kortere uitvoeringstijd. Eerder hebben we besproken hoe onvolledig benutte warps bijdragen aan thread-divergentie. Een andere oorzaak van divergente threads binnen een warp is het verschil in computationele belasting tussen threads. Bijvoorbeeld, als een rode thread meer rekentijd vereist dan een groene of blauwe thread, ontstaat er een vertraging binnen de warp. Dit wordt duidelijk wanneer we de berekeningstijden vergelijken: warps met een mix van rode, groene en blauwe threads presteren minder efficiënt dan warps waarin alle threads dezelfde kleur hebben (bijvoorbeeld alleen rode, alleen groene of alleen blauwe threads).



grafiek 3: Uitvoeringstijd (ms) als functie van het aantal threads (tot 32)

Code Warps loop

```
#include <stdio.h>
#include <cuda_runtime.h>
#include <cstdint> // Data types
#include <iostream> // File operations
#include <fstream> // For file operations

#define M 941 // VR width
#define N 704 // VR height
#define C 3 // Colors
#define OFFSET 15 // Header length

__global__ void image_invert(uint8_t* A, int lengte) {
       for (int i = blockIdx.x * blockDim.x + threadIdx.x; i < lengte; i += blockDim.x * gridDim.x)
           {
       A[i] = 255 - A[i];
    }
}

uint8_t* get_image_array() {
    FILE *imageFile;
    imageFile = fopen("./input_image.ppm", "rb");
    if (imageFile == NULL) {
        perror("ERROR: Cannot open input file");
        exit(EXIT_FAILURE);
    }

    uint8_t* image_array = (uint8_t*)malloc(M * N * C * sizeof(uint8_t) + OFFSET);
    fread(image_array, sizeof(uint8_t), M * N * C * sizeof(uint8_t) + OFFSET, imageFile);
    fclose(imageFile);

    return image_array + OFFSET;
}

void save_image_array(uint8_t* image_array) {
    FILE *imageFile;
    imageFile = fopen("./output_image.ppm", "wb");
    if (imageFile == NULL) {
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    fprintf(imageFile, "P6\n"); // P6 filetype
    fprintf(imageFile, "%d %d\n", M, N); // dimensions
    fprintf(imageFile, "255\n"); // Max pixel
    fwrite(image_array, 1, M * N * C, imageFile);
    fclose(imageFile);
}

int main() {
    uint8_t* image_array = get_image_array();
    uint8_t *d_image_array;

    int lengte = M * N * C;

    cudaMalloc(&d_image_array, lengte * sizeof(uint8_t));
    cudaMemcpy(d_image_array, image_array, lengte * sizeof(uint8_t), cudaMemcpyHostToDevice);

    std::ofstream outFile("result_warps.csv");
    outFile << "ThreadsPerBlock,Time(ms)\n";

    for (int threadsPerBlock = 1; threadsPerBlock <= 1023; threadsPerBlock++) {
        int blocksPerGrid = (lengte + threadsPerBlock - 1) / threadsPerBlock;
```

```
62
63          cudaEvent_t start, stop;
64          cudaEventCreate(&start);
65          cudaEventCreate(&stop);
66          cudaEventRecord(start);
67
68          image_invert<<<blocksPerGrid, threadsPerBlock>>>(d_image_array, lengte);
69          cudaDeviceSynchronize();
70
71          cudaEventRecord(stop);
72          cudaEventSynchronize(stop);
73          float milliseconds;
74          cudaEventElapsedTime(&milliseconds, start, stop);
75
76          outFile << threadsPerBlock << "," << milliseconds << "\n";
77
78          cudaEventDestroy(start);
79          cudaEventDestroy(stop);
80      }
81
82      outFile.close();
83
84      cudaMemcpy(image_array, d_image_array, lengte * sizeof(uint8_t), cudaMemcpyDeviceToHost);
85      save_image_array(image_array);
86
87      cudaFree(d_image_array);
88
89      return 0;
90  }
```

Code basic divergence

```
#include <stdio.h>
#include <cuda_runtime.h>
#include <cstdint> // Data types
#include <iostream> // File operations
#include <fstream> // For file operations

#define M 941 // VR width
#define N 704 // VR height
#define C 3 // Colors
#define OFFSET 15 // Header length

__global__ void image(uint8_t* A, int length) {
    for (int i = blockIdx.x * blockDim.x + threadIdx.x; i < length; i += blockDim.x * gridDim.x) {
        if (i%3 == 0){
            A[i] = (A[i] % 25) * 10;
        }
        else{
            A[i] = 255 - A[i];
        }
    }
}

uint8_t* get_image_array() {
    FILE *imageFile;
    imageFile = fopen("./input_image.ppm", "rb");
    if (imageFile == NULL) {
        perror("ERROR: Cannot open input file");
        exit(EXIT_FAILURE);
    }

    uint8_t* image_array = (uint8_t*)malloc(M * N * C * sizeof(uint8_t) + OFFSET);
    fread(image_array, sizeof(uint8_t), M * N * C * sizeof(uint8_t) + OFFSET, imageFile);
    fclose(imageFile);

    return image_array + OFFSET;
}

void save_image_array(uint8_t* image_array) {
    FILE *imageFile;
    imageFile = fopen("./output_image.ppm", "wb");
    if (imageFile == NULL) {
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    fprintf(imageFile, "P6\n"); // P6 filetype
    fprintf(imageFile, "%d %d\n", M, N); // dimensions
    fprintf(imageFile, "255\n"); // Max pixel
    fwrite(image_array, 1, M * N * C, imageFile);
    fclose(imageFile);
}

int main() {
    uint8_t* image_array = get_image_array();
    uint8_t *d_image_array;

    int length = M * N * C;

    cudaMalloc(&d_image_array, length * sizeof(uint8_t));
    cudaMemcpy(d_image_array, image_array, length * sizeof(uint8_t), cudaMemcpyHostToDevice);

    std::ofstream outFile("divergence_basic.csv");
```

```
    outFile << "ThreadsPerBlock,Time(ms)\n";

    for (int threadsPerBlock = 1; threadsPerBlock <= 2047; threadsPerBlock++) {
        int blocksPerGrid = ((length + threadsPerBlock - 1) / threadsPerBlock);

        cudaEvent_t start, stop;
        cudaEventCreate(&start);
        cudaEventCreate(&stop);
        cudaEventRecord(start);

        image<<<blocksPerGrid, threadsPerBlock>>>(d_image_array, length);
        cudaDeviceSynchronize();

        cudaEventRecord(stop);
        cudaEventSynchronize(stop);
        float milliseconds;
        cudaEventElapsedTime(&milliseconds, start, stop);

        outFile << threadsPerBlock << "," << milliseconds << "\n";

        cudaEventDestroy(start);
        cudaEventDestroy(stop);
    }

    outFile.close();

    cudaMemcpy(image_array, d_image_array, length * sizeof(uint8_t), cudaMemcpyDeviceToHost);
    save_image_array(image_array);

    cudaFree(d_image_array);

    return 0;
}
```

Code thread divergence met striding

```
#include <stdio.h>
#include <cuda_runtime.h>
#include <cstdint> // Data types
#include <iostream> // File operations
#include <fstream> // For file operations

#define M 941 // VR width
#define N 704 // VR height
#define C 3 // Colors
#define OFFSET 15 // Header length

__global__ void image(uint8_t* A, int length) {
    for (int i = (blockIdx.x * blockDim.x + threadIdx.x)*3; i < length; i += (blockDim.x * gridDim.x)
         *3) {
        A[i] = (A[i] % 25) * 10;
        A[i + 1] = 255 - A[i + 1];
        A[i + 2] = 255 - A[i + 2];
    }
}

uint8_t* get_image_array() {
    FILE *imageFile;
    imageFile = fopen("./input_image.ppm", "rb");
    if (imageFile == NULL) {
        perror("ERROR: Cannot open input file");
        exit(EXIT_FAILURE);
    }

    uint8_t* image_array = (uint8_t*)malloc(M * N * C * sizeof(uint8_t) + OFFSET);
    fread(image_array, sizeof(uint8_t), M * N * C * sizeof(uint8_t) + OFFSET, imageFile);
    fclose(imageFile);

    return image_array + OFFSET;
}

void save_image_array(uint8_t* image_array) {
    FILE *imageFile;
    imageFile = fopen("./output_image.ppm", "wb");
    if (imageFile == NULL) {
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    fprintf(imageFile, "P6\n"); // P6 filetype
    fprintf(imageFile, "%d %d\n", M, N); // dimensions
    fprintf(imageFile, "255\n"); // Max pixel
    fwrite(image_array, 1, M * N * C, imageFile);
    fclose(imageFile);
}

int main() {
    uint8_t* image_array = get_image_array();
    uint8_t *d_image_array;

    int length = M * N * C;

    cudaMalloc(&d_image_array, length * sizeof(uint8_t));
    cudaMemcpy(d_image_array, image_array, length * sizeof(uint8_t), cudaMemcpyHostToDevice);

    std::ofstream outFile("divergence_RGB_1_Thread_striding.csv");
    outFile << "ThreadsPerBlock,Time(ms)\n";
    int threadsPerBlock = 1023;
```

```
62
63      for (int threads = 1; threads <= 100000; threads++) {
64
65          int blocksPerGrid = (threads + threadsPerBlock - 1) / threadsPerBlock;
66
67          cudaEvent_t start, stop;
68          cudaEventCreate(&start);
69          cudaEventCreate(&stop);
70          cudaEventRecord(start);
71
72          image<<<blocksPerGrid, threadsPerBlock>>>(d_image_array, length);
73          cudaDeviceSynchronize();
74
75          cudaEventRecord(stop);
76          cudaEventSynchronize(stop);
77          float milliseconds;
78          cudaEventElapsedTime(&milliseconds, start, stop);
79
80          outFile << threadsPerBlock << "," << milliseconds << "\n";
81
82          cudaEventDestroy(start);
83          cudaEventDestroy(stop);
84      }
85
86      outFile.close();
87
88      cudaMemcpy(image_array, d_image_array, length * sizeof(uint8_t), cudaMemcpyDeviceToHost);
89      save_image_array(image_array);
90
91      cudaFree(d_image_array);
92
93      return 0;
94  }
```

Code thread divergence RGB in volgorde

```
#include <stdio.h>
#include <cuda_runtime.h>
#include <cstdint> // Data types
#include <iostream> // File operations
#include <fstream> // For file operations

#define M 941 // VR width
#define N 704 // VR height
#define C 3 // Colors
#define OFFSET 15 // Header length

__global__ void image(uint8_t* A, int length) {
    int i = (blockIdx.x * blockDim.x + threadIdx.x)*3;
    if ((i) < length) {
        A[i] = (A[i] % 25) * 10;
    }
    else if (((i+1)-length)< length){
        A[(i+1)-(length)] = 255 - A[(i+1)-(length)];
    }
    else if (((i+2)-(length*2))< length){
        A[(i+2)-(2*length)] = 255 - A[(i+2)-(2*length)];
    }
}

uint8_t* get_image_array() {
    FILE *imageFile;
    imageFile = fopen("./input_image.ppm", "rb");
    if (imageFile == NULL) {
        perror("ERROR: Cannot open input file");
        exit(EXIT_FAILURE);
    }

    uint8_t* image_array = (uint8_t*)malloc(M * N * C * sizeof(uint8_t) + OFFSET);
    fread(image_array, sizeof(uint8_t), M * N * C * sizeof(uint8_t) + OFFSET, imageFile);
    fclose(imageFile);

    return image_array + OFFSET;
}

void save_image_array(uint8_t* image_array) {
    FILE *imageFile;
    imageFile = fopen("./output_image.ppm", "wb");
    if (imageFile == NULL) {
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    fprintf(imageFile, "P6\n"); // P6 filetype
    fprintf(imageFile, "%d %d\n", M, N); // dimensions
    fprintf(imageFile, "255\n"); // Max pixel
    fwrite(image_array, 1, M * N * C, imageFile);
    fclose(imageFile);
}

int main() {
    uint8_t* image_array = get_image_array();
    uint8_t *d_image_array;

    int length = M * N * C;

    cudaMalloc(&d_image_array, length * sizeof(uint8_t));
    cudaMemcpy(d_image_array, image_array, length * sizeof(uint8_t), cudaMemcpyHostToDevice);
```

```
63
64      std::ofstream outFile("divergence_basic.csv");
65      outFile << "ThreadsPerBlock,Time(ms)\n";
66
67      for (int threadsPerBlock = 1; threadsPerBlock <= 2047; threadsPerBlock++) {
68          int blocksPerGrid = ((length + threadsPerBlock - 1) / threadsPerBlock);
69
70          cudaEvent_t start, stop;
71          cudaEventCreate(&start);
72          cudaEventCreate(&stop);
73          cudaEventRecord(start);
74
75          image<<<blocksPerGrid, threadsPerBlock>>>(d_image_array, length);
76          cudaDeviceSynchronize();
77
78          cudaEventRecord(stop);
79          cudaEventSynchronize(stop);
80          float milliseconds;
81          cudaEventElapsedTime(&milliseconds, start, stop);
82
83          outFile << threadsPerBlock << "," << milliseconds << "\n";
84
85          cudaEventDestroy(start);
86          cudaEventDestroy(stop);
87      }
88
89      outFile.close();
90
91      cudaMemcpy(image_array, d_image_array, length * sizeof(uint8_t), cudaMemcpyDeviceToHost);
92      save_image_array(image_array);
93
94      cudaFree(d_image_array);
95
96      return 0;
97  }
```